Prompting best practices

Comprehensive guide to prompt engineering techniques for Claude's latest models, covering clarity, examples, XML structuring, thinking, and agentic systems.

This is the reference for prompt engineering with current Claude models, including Claude Fable 5.1, Claude Mythos 5.1, Claude Fable 5, Claude Mythos 5, Claude Opus 5.5, Claude Opus 5, Claude Opus 4.8, Claude Opus 4.7, Claude Opus 4.6, Claude Sonnet 5.5, Claude Sonnet 5, Claude Sonnet 4.6, Claude Haiku 5.5, and Claude Haiku 4.5. The page is organized in three parts:

- **Model-specific guidance** first: where a single model behaves differently and what to change in your prompt.
- **Techniques for all current models** after that: general principles, output and formatting, tool use, thinking, and agentic systems.
- **Migration considerations** last, for prompts moving from earlier generations.

> For an overview of model capabilities, see the models overview. For Claude Fable 5.1 capabilities and API changes, see What's new in Claude Fable 5.1. For migration guidance, see the Migration guide. For Claude Opus 5.5, see What's new in Claude Opus 5.5. For Claude Sonnet 5.5, see What's new in Claude Sonnet 5.5. For Claude Haiku 5.5, see What's new in Claude Haiku 5.5.

Model-specific guidance

Each of these models has its own prompting page. Read the one for your model first, then the techniques that follow.

Model	Guide	What's different
Claude Fable 5.1 and Claude Mythos 5.1	Prompting Claude Fable 5.1	Differences from Claude Fable 5: effort levels, finishing long tasks, user-facing progress updates, passing thinking blocks back unchanged, tool-call batching in agent loops, search triggering at low effort, formatting, and writing density.

Model	Guide	What's different
Claude Fable 5 and Claude Mythos 5	Prompting Claude Fable 5	Differences from Claude Opus 4.8: effort levels, instruction following, long-run progress claims, memory systems, and the `reasoning_extraction` refusal category.
Claude Sonnet 5.5	Prompting Claude Sonnet 5.5	Differences from Claude Sonnet 5: effort calibration, initiative and scope, running without up-front thinking, JSON output on reasoning tasks, user-facing progress updates, tool use in chat, mid-turn user messages, verification on coding tasks, tool-call handling, complex visual inputs, and safeguard refusals.
Claude Sonnet 5	Prompting Claude Sonnet 5	Differences from Claude Sonnet 4.6: response length, effort and thinking-depth calibration, tool use triggering, literal instruction following, and design and frontend defaults.
Claude Opus 5.5	Prompting Claude Opus 5.5	Differences from Claude Opus 5: effort calibration, prompts written for thinking disabled, user-facing progress updates, safeguard refusals, and tools for complex visual inputs.
Claude Opus 5	Prompting Claude Opus 5	Differences from prior Opus models: response length and verbosity, user-facing progress updates, written deliverable length, task scope and over-verification, subagent control, and self-correction.
Claude Opus 4.8	Prompting Claude Opus 4.8	Response length, effort and thinking-depth calibration, tool use triggering, literal instruction following, subagent control, and design and frontend defaults.
Claude Haiku 5.5	Prompting Claude Haiku 5.5	Differences from Claude Haiku 4.5: effort levels, search, JSON output with your own tools, early stopping in long agent prompts, verification on coding tasks, mid-turn user messages, system prompt adherence in chatbots, and safeguard refusals.

General principles

The techniques in this section and the sections that follow apply to current Claude models, including Claude Fable 5.1, Claude Mythos 5.1, Claude Fable 5, and Claude Mythos 5. Where a technique names a specific model, treat it as measured on that model and re-check it against your own evals before applying it to another.

Be clear and direct

Claude responds well to clear, explicit instructions. Being specific about your desired output can help enhance results. If you want "above and beyond" behavior, explicitly request it rather than relying on the model to infer this from vague prompts.

Think of Claude as a brilliant but new employee who lacks context on your norms and workflows. The more precisely you explain what you want, the better the result.

Golden rule: Show your prompt to a colleague with minimal context on the task and ask them to follow it. If they'd be confused, Claude will be too.

- Be specific about the desired output format and constraints.
- Provide instructions as sequential steps using numbered lists or bullet points when the order or completeness of steps matters.

Example: Creating an analytics dashboard

Less effective:

```
Create an analytics dashboard
```

More effective:

```
Create an analytics dashboard. Include as many relevant features and
interactions as possible. Go beyond the basics to create a fully-featured
implementation.
```

Add context to improve performance

Providing context or motivation behind your instructions, such as explaining to Claude why such behavior is important, can help Claude better understand your goals and deliver more targeted responses.

Example: Formatting preferences

Less effective:

```
NEVER use ellipses
```

More effective:

```
Your response will be read aloud by a text-to-speech engine, so never use
ellipses since the text-to-speech engine will not know how to pronounce them.
```

Claude is smart enough to generalize from the explanation.

Use examples effectively

Examples are one of the most reliable ways to steer Claude's output format, tone, and structure. A few well-crafted examples (known as few-shot or multishot prompting) improve accuracy and consistency.

When adding examples, make them:

- **Relevant:** Mirror your actual use case closely.
- **Diverse:** Cover edge cases and vary enough that Claude doesn't pick up unintended patterns.
- **Structured:** Wrap examples in `<example>` tags (multiple examples in `<examples>` tags) so Claude can distinguish them from instructions.

> Include 3–5 examples for best results. You can also ask Claude to evaluate your examples for relevance and diversity, or to generate additional ones based on your initial set.

Structure prompts with XML tags

XML tags help Claude parse complex prompts unambiguously, especially when your prompt mixes instructions, context, examples, and variable inputs. Wrapping each type of content in its own tag (for example, `<instructions>`, `<context>`, `<input>`) reduces misinterpretation.

Best practices:

- Use consistent, descriptive tag names across your prompts.
- Nest tags when content has a natural hierarchy (documents inside `<documents>`, each inside `<document index="n">`).

Give Claude a role

Setting a role in the system prompt focuses Claude's behavior and tone for your use case. Even a single sentence makes a difference:

Python

```python
client = anthropic.Anthropic()

message = client.messages.create(
    model="claude-opus-5-5",
    max_tokens=1024,
    system="You are a helpful coding assistant specializing in Python.",
    messages=[
        {"role": "user", "content": "How do I sort a list of dictionaries by key?"}
    ],
)

print(message.content)
```

Long context prompting

When working with large documents or data-rich inputs (20k+ tokens), structure your prompt carefully to get the best results:

- **Put longform data at the top:** Place your long documents and inputs near the top of your prompt, above your query, instructions, and examples. This improves performance across all models.

  > Queries at the end can improve response quality by up to 30 percent in tests, especially with complex, multidocument inputs.

- **Structure document content and metadata with XML tags:** When using multiple documents, wrap each document in `<document>` tags with `<document_content>` and `<source>` (and other metadata) subtags for clarity.

  Example multidocument structure

```xml
<documents>
  <document index="1">
    <source>annual_report_2023.pdf</source>
    <document_content>
      {{ANNUAL_REPORT}}
    </document_content>
  </document>
  <document index="2">
```

```
    <source>competitor_analysis_q2.xlsx</source>
    <document_content>
      {{COMPETITOR_ANALYSIS}}
    </document_content>
  </document>
</documents>

Analyze the annual report and competitor analysis. Identify strategic
advantages and recommend Q3 focus areas.
```

- **Ground responses in quotes:** For long document tasks, ask Claude to quote relevant parts of the documents first before carrying out its task. This helps Claude focus on the relevant content and ignore the rest of the document.

  Example quote extraction

```
You are an AI physician's assistant. Your task is to help doctors diagnose
possible patient illnesses.

<documents>
  <document index="1">
    <source>patient_symptoms.txt</source>
    <document_content>
      {{PATIENT_SYMPTOMS}}
    </document_content>
  </document>
  <document index="2">
    <source>patient_records.txt</source>
    <document_content>
      {{PATIENT_RECORDS}}
    </document_content>
  </document>
  <document index="3">
    <source>patient01_appt_history.txt</source>
    <document_content>
      {{PATIENT01_APPOINTMENT_HISTORY}}
    </document_content>
  </document>
</documents>

Find quotes from the patient records and appointment history that are
relevant to diagnosing the patient's reported symptoms. Place these in
<quotes> tags. Then, based on these quotes, list all information that
would help the doctor diagnose the patient's symptoms. Place your
diagnostic information in <info> tags.
```

Model self-knowledge

If you would like Claude to identify itself correctly in your application or use specific API strings:

Sample prompt for model identity

```
The assistant is Claude, created by Anthropic. The current model is Claude Opus
5.5.
```

For LLM-powered apps that need to specify model strings:

Sample prompt for model string

```
When an LLM is needed, please default to Claude Opus 5.5 unless the user
requests
otherwise. The exact model string for Claude Opus 5.5 is claude-opus-5-5.
```

Output and formatting

Communication style and verbosity

Claude's latest models have a more concise and natural communication style compared to previous models:

- **More direct and grounded:** Provides fact-based progress reports rather than self-celebratory updates
- **More conversational:** Slightly more fluent and colloquial, less machine-like
- **Less verbose:** May skip detailed summaries for efficiency unless prompted otherwise

This means Claude may skip verbal summaries after tool calls, jumping directly to the next action. If you prefer more visibility into its work:

Sample prompt

```
After completing a task that involves tool use, provide a quick summary of the
work you've done.
```

Claude Opus 5 is an exception on verbosity: its default user-facing responses run longer than prior models', and raising or lowering effort does not reliably change visible response length. Prompt explicitly for conciseness instead. See Prompting Claude Opus 5 for a sample instruction. Claude Fable 5.1 has the opposite tendency during agentic work: it writes fewer

user-facing updates between tool calls. Ask for progress text explicitly, and remove any instruction telling it to keep that text brief. See Ask for user-facing progress updates.

Control the format of responses

There are a few particularly effective ways to steer output formatting:

1. **Tell Claude what to do instead of what not to do**
   - Instead of: "Do not use markdown in your response"
   - Try: "Your response should be composed of smoothly flowing prose paragraphs."
2. **Use XML format indicators**
   - Try: "Write the prose sections of your response in <smoothly_flowing_prose_paragraphs> tags."
3. **Match your prompt style to the desired output**

   The formatting style used in your prompt may influence Claude's response style. If you are still experiencing steerability issues with output formatting, try matching your prompt style to your desired output style as closely as possible. For example, removing markdown from your prompt can reduce the volume of markdown in the output.
4. **Use detailed prompts for specific formatting preferences**

   For more control over markdown and formatting usage, provide explicit guidance:

Sample prompt to minimize markdown

```
<avoid_excessive_markdown_and_bullet_points>
When writing reports, documents, technical explanations, analyses, or any
long-form
content, write in clear, flowing prose using complete paragraphs and sentences.
Use
standard paragraph breaks for organization and reserve markdown primarily for
`inline
code`, code blocks (```...```), and simple headings (## and ###). Avoid using
**bold**
and *italics*.

DO NOT use ordered lists (1. ...) or unordered lists (*) unless: a) you're
presenting
truly discrete items where a list format is the best option, or b) the user
explicitly
requests a list or ranking

Instead of listing items with bullets or numbers, incorporate them naturally
into
sentences. This guidance applies especially to technical writing. Using prose
instead of
```

```
excessive formatting will improve user satisfaction. NEVER output a series of
overly
short bullet points.

Your goal is readable, flowing text that guides the reader naturally through
ideas
rather than fragmenting information into isolated points.
</avoid_excessive_markdown_and_bullet_points>
```

Claude Fable 5.1 already formats less than earlier models, so on that model a block like this can suppress structure the content needs. Remove it, or replace it with the shorter rule in Formatting in chat.

LaTeX output

Claude's latest models default to LaTeX for mathematical expressions, equations, and technical explanations. If you prefer plain text, add the following instructions to your prompt:

Sample prompt

```
Format your response in plain text only. Do not use LaTeX, MathJax, or any
markup
notation such as \( \), $, or \frac{}{}. Write all math expressions using
standard text
characters (e.g., "/" for division, "*" for multiplication, and "^" for
exponents).
```

Document creation

Claude's latest models create presentations, animations, and visual documents with strong instruction following, and usually produce usable output on the first try.

For best results with document creation:

Sample prompt

```
Create a professional presentation on [topic]. Include thoughtful design
elements,
visual hierarchy, and engaging animations where appropriate.
```

Migrating away from prefilled responses

Starting with Claude 4.6 models and Claude Mythos Preview, prefilled responses (providing a partial assistant message for Claude to continue from) on the last assistant turn are no longer supported. Requests with prefilled assistant messages to these models return a 400 error.

Model intelligence and instruction following have advanced such that most use cases of prefill no longer require it. Earlier models continue to support prefills, and adding assistant messages elsewhere in the conversation is not affected.

Here are common prefill scenarios and how to migrate away from them:

Controlling output formatting

Prefills have been used to force specific output formats like JSON/YAML, classification, and similar patterns where the prefill constrains Claude to a particular structure.

Migration: The Structured Outputs feature is designed specifically to constrain Claude's responses to follow a given schema. Try asking the model to conform to your output structure first, as newer models can reliably match complex schemas when told to, especially if implemented with retries. For classification tasks, use either tools with an enum field containing your valid labels or structured outputs.

Eliminating preambles

Prefills like `Here is the requested summary:\n` were used to skip introductory text.

Migration: Use direct instructions in the system prompt: "Respond directly without preamble. Do not start with phrases like 'Here is...', 'Based on...', etc." Alternatively, direct the model to output within XML tags, use structured outputs, or use tool calling. If the occasional preamble slips through, strip it in post-processing.

Avoiding bad refusals

Prefills were used to steer around unnecessary refusals.

Migration: Claude is much better at appropriate refusals now. Clear prompting within the `user` message without prefill should be sufficient.

Continuations

Prefills were used to continue partial completions, resume interrupted responses, or pick up where a previous generation left off.

Migration: Move the continuation to the user message, and include the final text from the interrupted response: "Your previous response was interrupted and ended with `[previous_response]`. Continue from where you left off." If this is part of error-handling or incomplete-response-handling and there is no UX penalty, retry the request.

Context hydration and role consistency

Prefills were used to periodically ensure refreshed or injected context.

Migration: For very long conversations, inject what were previously prefilled-assistant reminders into the user turn. If context hydration is part of a more complex agentic system, consider hydrating through tools (expose or encourage use of tools containing context based on heuristics such as number of turns) or during context compaction.

Tool use

Tool usage

Claude's latest models are trained for precise instruction following and benefit from explicit direction to use specific tools. If you say "can you suggest some changes," Claude will sometimes provide suggestions rather than implementing them, even if making changes might be what you intended. To learn how to define tools and troubleshoot tool triggering, see Tool use with Claude.

For Claude to take action, be more explicit:

Example: Explicit instructions

Less effective (Claude will only suggest):

```
Can you suggest some changes to improve this function?
```

More effective (Claude will make the changes):

```
Change this function to improve its performance.
```

Or:

```
Make these edits to the authentication flow.
```

To make Claude more proactive about taking action by default, you can add this to your system prompt:

Sample prompt for proactive action

```
<default_to_action>
By default, implement changes rather than only suggesting them. If the user's
intent is
unclear, infer the most useful likely action and proceed, using tools to
discover any
missing details instead of guessing. Try to infer the user's intent about
whether a tool
call (e.g., file edit or read) is intended or not, and act accordingly.
</default_to_action>
```

On the other hand, if you want the model to be more hesitant by default, less prone to jumping straight into implementations, and only take action if requested, you can steer this behavior with a prompt like the following:

Sample prompt for conservative action

```
<do_not_act_before_instructions>
Do not jump into implementation or change files unless clearly instructed to
make
changes. When the user's intent is ambiguous, default to providing information,
doing
research, and providing recommendations rather than taking action. Only proceed
with
edits, modifications, or implementations when the user explicitly requests
them.
</do_not_act_before_instructions>
```

Claude Opus 4.5 and Claude Opus 4.6 are also more responsive to the system prompt than previous models. If your prompts were designed to reduce undertriggering on tools or skills, these models may now overtrigger. The fix is to dial back any aggressive language. Where you might have said "CRITICAL: You MUST use this tool when...", you can use more normal prompting like "Use this tool when...".

Optimize parallel tool calling

Claude's latest models run independent tool calls in parallel. These models will:

- Run multiple speculative searches during research
- Read several files at once to build context faster
- Run bash commands in parallel (which can even bottleneck system performance)

This behavior is steerable. While the model has a high success rate in parallel tool calling without prompting, you can boost this to ~100% or adjust the aggression level:

Sample prompt for maximum parallel efficiency

```
<use_parallel_tool_calls>
If you intend to call multiple tools and there are no dependencies between the
tool
calls, make all of the independent tool calls in parallel. Prioritize calling
tools
simultaneously whenever the actions can be done in parallel rather than
sequentially.
For example, when reading 3 files, run 3 tool calls in parallel to read all 3
files into
context at the same time. Maximize use of parallel tool calls where possible to
increase
speed and efficiency. However, if some tool calls depend on previous calls to
inform
dependent values like the parameters, do NOT call these tools in parallel and
instead
call them sequentially. Never use placeholders or guess missing parameters in
tool
calls.
</use_parallel_tool_calls>
```

Sample prompt to reduce parallel execution

```
Execute operations sequentially with brief pauses between each step to ensure
stability.
```

On Claude Fable 5.1 in long agent loops, send the parallel-calls instruction as a turn-scoped system message after each round of tool results. See Batch independent tool calls in agent loops.

Thinking and reasoning

Overthinking and excessive thoroughness

Claude Opus 4.6 does more upfront exploration than previous models, especially at higher `effort` settings. This initial work often helps to optimize the final results, but the model may gather extensive context or pursue multiple threads of research without being prompted. If your prompts previously encouraged the model to be more thorough, you should tune that guidance for Claude Opus 4.6:

- **Replace blanket defaults with more targeted instructions.** Instead of "Default to using [tool]," add guidance like "Use [tool] when it would enhance your understanding of the

problem."

- **Remove over-prompting.** Tools that undertriggered in previous models are likely to trigger appropriately now. Instructions like "If in doubt, use [tool]" will cause overtriggering.
- **Use effort as a fallback.** If Claude continues to be overly aggressive, use a lower setting for `effort`.

In some cases, Claude Opus 4.6 may think extensively, which can inflate thinking tokens and slow down responses. If this behavior is undesirable, you can add explicit instructions to constrain its reasoning, or you can lower the `effort` setting to reduce overall thinking and token usage.

Sample prompt

```
When you're deciding how to approach a problem, choose an approach and commit
to it.
Avoid revisiting decisions unless you encounter new information that directly
contradicts your reasoning. If you're weighing two approaches, pick one and see
it
through. You can always course-correct later if the chosen approach fails.
```

If you need a hard ceiling on thinking costs, extended thinking with a `budget_tokens` cap is still functional on Opus 4.6 and Sonnet 4.6 but is deprecated. On Claude 4.7 and later models, setting `budget_tokens` returns a 400 error. Prefer lowering the effort setting or using `max_tokens` as a hard limit with adaptive thinking.

Leverage thinking & interleaved thinking capabilities

Claude's latest models offer thinking capabilities that can be especially helpful for tasks involving reflection after tool use or complex multistep reasoning. You can guide its initial or interleaved thinking for better results.

Claude 4.6 and later models and Claude Mythos Preview use adaptive thinking (`thinking: {type: "adaptive"}`), where Claude dynamically determines when and how much to think. On Claude Fable 5.1, Claude Mythos 5.1, Claude Fable 5, Claude Mythos 5, and Claude Opus 5.5, thinking is always on and adaptive thinking is the only mode. Claude calibrates its thinking based on two factors: the `effort` parameter and query complexity. Higher effort elicits more thinking, and more complex queries do the same. On easier queries that don't require thinking, the model responds directly. In internal evaluations, adaptive thinking reliably drives better performance than extended thinking. Consider moving to adaptive thinking.

Use adaptive thinking for workloads that require agentic behavior such as multistep tool use, complex coding tasks, and long-horizon agent loops. Older models use manual extended

thinking with `budget_tokens`; see the per-model configuration table for which configuration each model accepts.

You can guide Claude's thinking behavior:

Example prompt

```
After receiving tool results, carefully reflect on their quality and determine
optimal
next steps before proceeding. Use your thinking to plan and iterate based on
this new
information, and then take the best next action.
```

The triggering behavior for adaptive thinking is promptable. If you find the model thinking more often than you'd like, which can happen with large or complex system prompts, add guidance to steer it:

Sample prompt

```
Thinking adds latency and should only be used when it will meaningfully improve
answer quality - typically for problems that require multistep reasoning. When
in
doubt, respond directly.
```

If you are migrating from extended thinking with `budget_tokens`, replace your thinking configuration and move budget control to `effort`. The following examples show the same request before and after the migration (see effort for the available levels and per-model availability):

Python

```python
# Before: extended thinking with a manual budget (older models)
client.messages.create(
    model="claude-sonnet-4-5-20250929",
    max_tokens=16000,
    thinking={"type": "enabled", "budget_tokens": 10000},
    messages=[{"role": "user", "content": "..."}],
)

# After: adaptive thinking with effort
client.messages.create(
    model="claude-opus-4-8",
    max_tokens=16000,
    thinking={"type": "adaptive"},
    output_config={"effort": "high"},
```

```
    messages=[{"role": "user", "content": "..."}],
)
```

If you are not using extended thinking, you don't need to add a `thinking` parameter. On Claude Opus 4.6 through Claude Opus 4.8 and Claude Sonnet 4.6, thinking is off when you omit the `thinking` parameter. On Claude Opus 5, Claude Sonnet 5.5, Claude Sonnet 5, and Claude Haiku 5.5, thinking is on by default when you omit the `thinking` parameter, so a response can begin with `thinking` blocks. Select content blocks by `type` rather than by position, and leave room in `max_tokens` for thinking. On Claude Opus 5 and Claude Haiku 5.5, you can disable it only at effort `high` or lower. On Claude Sonnet 5.5, the lowest thinking setting is `between_tools`, accepted at effort `high` or lower. On Claude Fable 5.1, Claude Mythos 5.1, Claude Fable 5, Claude Mythos 5, and Claude Opus 5.5, thinking is always on, regardless of whether you set the `thinking` parameter.

- **Prefer general instructions over prescriptive steps.** A prompt like "think thoroughly" often produces better reasoning than a hand-written step-by-step plan. Claude's reasoning frequently exceeds what a human would prescribe.
- **Multishot examples work with thinking.** Worked examples in your prompt shape how Claude approaches similar problems in its own thinking blocks. Present each example as a problem, the method to apply, and the expected answer.
- **Manual chain-of-thought (CoT) prompting as a fallback.** When thinking is off, you can still encourage step-by-step reasoning by asking Claude to think through the problem before it answers, and to put the final answer in `<answer>` tags so you can extract it. On Claude Fable 5.1, Claude Fable 5, Claude Opus 5.5, Claude Opus 5, and Claude Sonnet 5.5, rely on thinking instead, at a lower effort level if cost or latency matters: a prompt that asks the model to write out its reasoning, for example in `<thinking>` tags, may be declined. See Keep reasoning in thinking blocks.
- **Ask Claude to self-check.** Append something like "Before you finish, verify your answer against [test criteria]." This catches errors reliably, especially for coding and math. Claude Opus 5 is the exception: it verifies its own work well without explicit instruction, and verification instructions carried over from prompts tuned for earlier models can cause over-verification, adding tokens and latency. On Claude Opus 5, remove these instructions rather than rewriting them. See Task scope and over-verification.

When extended thinking is disabled, Claude Opus 4.5 is particularly sensitive to the word "think" and its variants. Consider using alternatives like "consider," "evaluate," or "reason through" in those cases.

For more information on thinking capabilities, see Thinking and Steering thinking.

Agentic systems

Long-horizon reasoning and state tracking

Claude's latest models handle long-horizon reasoning tasks with strong state tracking. Claude maintains orientation across extended sessions by focusing on incremental progress, making steady advances on a few things at a time rather than attempting everything at once. This capability especially emerges over multiple context windows or task iterations, where Claude can work on a complex task, save the state, and continue with a fresh context window.

Context awareness and multiwindow workflows

Claude Sonnet 5, Claude Sonnet 4.6, Claude Sonnet 4.5 (deprecated), and Claude Haiku 4.5 feature context awareness, enabling the model to track its remaining context window (that is, its "token budget") throughout a conversation. This enables Claude to execute tasks and manage context more effectively by understanding how much space it has to work.

Managing context limits:

If you are using Claude in an agent harness that compacts context or allows saving context to external files (like in Claude Code), consider adding this information to your prompt so Claude can behave accordingly. Otherwise, Claude may sometimes naturally try to wrap up work as it approaches the context limit. The following is an example prompt:

Sample prompt

```
Your context window will be automatically compacted as it approaches its limit,
allowing
you to continue working indefinitely from where you left off. Therefore, do not
stop
tasks early due to token budget concerns. As you approach your token budget
limit, save
your current progress and state to memory before the context window refreshes.
Always be
as persistent and autonomous as possible and complete tasks fully, even if the
end of
your budget is approaching. Never artificially stop any task early regardless
of the
context remaining.
```

The memory tool pairs well with context awareness for managing context transitions.

Workflows across multiple context windows

For tasks spanning multiple context windows:

1. **Use a different prompt for the very first context window:** Use the first context window to set up a framework (write tests, create setup scripts), then use future context windows to iterate on a todo-list.
2. **Have the model write tests in a structured format:** Ask Claude to create tests before starting work and keep track of them in a structured format (for example, `tests.json`). This leads to better long-term ability to iterate. Remind Claude of the importance of tests: "It is unacceptable to remove or edit tests because this could lead to missing or buggy functionality."
3. **Set up quality of life tools:** Encourage Claude to create setup scripts (for example, `init.sh`) to gracefully start servers, run test suites, and linters. This prevents repeated work when continuing from a fresh context window.
4. **Starting fresh versus compacting:** When a context window is cleared, consider starting with a brand new context window rather than using compaction. Claude's latest models are extremely effective at discovering state from the local filesystem. In some cases, you may want to take advantage of this over compaction. Be prescriptive about how it should start:
    - "Call pwd; you can only read and write files in this directory."
    - "Review progress.txt, tests.json, and the git logs."
    - "Manually run through a fundamental integration test before moving on to implementing new features."
5. **Provide verification tools:** As the length of autonomous tasks grows, Claude needs to verify correctness without continuous human feedback. Tools that let Claude verify UI work are helpful, such as the computer use tool, the browser use tool, or a browser automation MCP server.
6. **Encourage complete usage of context:** Prompt Claude to efficiently complete components before moving on:

Sample prompt

```
This is a very long task, so it may be beneficial to plan out your work
clearly. It's
encouraged to spend your entire output context working on the task - just make
sure you
don't run out of context with significant uncommitted work. Continue working
systematically until you have completed this task.
```

State management best practices

- **Use structured formats for state data:** When tracking structured information (like test results or task status), use JSON or other structured formats to help Claude understand schema requirements.

- **Use unstructured text for progress notes:** Freeform progress notes work well for tracking general progress and context.
- **Use git for state tracking:** Git provides a log of what's been done and checkpoints that can be restored. Claude's latest models perform especially well in using git to track state across multiple sessions.
- **Emphasize incremental progress:** Explicitly ask Claude to keep track of its progress and focus on incremental work.

Example: State tracking

tests.json

```
{
  "tests": [
    { "id": 1, "name": "authentication_flow", "status": "passing" },
    { "id": 2, "name": "user_management", "status": "failing" },
    { "id": 3, "name": "api_endpoints", "status": "not_started" }
  ],
  "total": 200,
  "passing": 150,
  "failing": 25,
  "not_started": 25
}
```

```
// Progress notes (progress.txt)
Session 3 progress:
- Fixed authentication token validation
- Updated user model to handle edge cases
- Next: investigate user_management test failures (test #2)
- Note: Do not remove tests as this could lead to missing functionality
```

Balancing autonomy and safety

Without guidance, Claude Opus 4.6 may take actions that are difficult to reverse or affect shared systems, such as deleting files, force-pushing, or posting to external services. If you

want Claude Opus 4.6 to confirm before taking potentially risky actions, add guidance to your prompt:

Sample prompt

```
Consider the reversibility and potential impact of your actions. You are
encouraged to
take local, reversible actions like editing files or running tests, but for
actions that
are hard to reverse, affect shared systems, or could be destructive, ask the
user before
proceeding.

Examples of actions that warrant confirmation:
- Destructive operations: deleting files or branches, dropping database tables,
rm -rf
- Hard to reverse operations: git push --force, git reset --hard, amending
published commits
- Operations visible to others: pushing code, commenting on PRs/issues, sending
messages, modifying shared infrastructure

When encountering obstacles, do not use destructive actions as a shortcut. For
example,
don't bypass safety checks (e.g. --no-verify) or discard unfamiliar files that
may be
in-progress work.
```

Research and information gathering

Claude's latest models can find and synthesize information from multiple sources effectively. For optimal research results:

1. **Provide clear success criteria:** Define what constitutes a successful answer to your research question.
2. **Encourage source verification:** Ask Claude to verify information across multiple sources.
3. **For complex research tasks, use a structured approach:**

Sample prompt for complex research

```
Search for this information in a structured way. As you gather data, develop
several
competing hypotheses. Track your confidence levels in your progress notes to
improve
calibration. Regularly self-critique your approach and plan. Update a
hypothesis tree or
```

```
research notes file to persist information and provide transparency. Break down
this
complex research task systematically.
```

This structured approach helps Claude work through large corpora methodically and iteratively critique its findings.

Subagent orchestration

Claude's latest models orchestrate subagents natively. These models can recognize when tasks would benefit from delegating work to specialized subagents and do so proactively without requiring explicit instruction.

To take advantage of this behavior:

1. **Ensure well-defined subagent tools:** Have subagent tools available and described in tool definitions.
2. **Let Claude orchestrate naturally:** Claude will delegate appropriately without explicit instruction.
3. **Watch for overuse:** Claude Opus 4.6 has a strong predilection for subagents and may spawn them in situations where a simpler, direct approach would suffice. For example, the model may spawn subagents for code exploration when a direct grep call is faster and sufficient. Claude Opus 5 also delegates to subagents more readily than prior models; see Controlling subagent spawning for guidance and a sample damping prompt.

If you're seeing excessive subagent use, add explicit guidance about when subagents are and aren't warranted:

Sample prompt for subagent usage

```
Use subagents when tasks can run in parallel, require isolated context, or
involve
independent workstreams that don't need to share state. For simple tasks,
sequential
operations, single-file edits, or tasks where you need to maintain context
across steps,
work directly rather than delegating.
```

Chain complex prompts

With adaptive thinking and subagent orchestration, Claude handles most multistep reasoning internally. Explicit prompt chaining (breaking a task into sequential API calls) is still useful when you need to inspect intermediate outputs or enforce a specific pipeline structure.

The most common chaining pattern is **self-correction:** generate a draft → have Claude review it against criteria → have Claude refine based on the review. Each step is a separate API call so you can log, evaluate, or branch at any point.

Reduce file creation in agentic coding

Claude's latest models may sometimes create new files for testing and iteration purposes, particularly when working with code. This approach allows Claude to use files, especially Python scripts, as a 'temporary scratchpad' before saving its final output. Using temporary files can improve outcomes particularly for agentic coding use cases.

If you'd prefer to minimize net new file creation, you can instruct Claude to clean up after itself:

Sample prompt

```
If you create any temporary new files, scripts, or helper files for iteration,
clean up
these files by removing them at the end of the task.
```

Overeagerness

Claude Opus 4.5 and Claude Opus 4.6 have a tendency to overengineer by creating extra files, adding unnecessary abstractions, or building in flexibility that wasn't requested. If you're seeing this undesired behavior, add specific guidance to keep solutions minimal.

For example:

Sample prompt to minimize overengineering

```
Avoid over-engineering. Only make changes that are directly requested or
clearly
necessary. Keep solutions simple and focused:

- Scope: Don't add features, refactor code, or make "improvements" beyond what
was
asked. A bug fix doesn't need surrounding code cleaned up. A simple feature
doesn't need
extra configurability.

- Documentation: Don't add docstrings, comments, or type annotations to code
you didn't
change. Only add comments where the logic isn't self-evident.

- Defensive coding: Don't add error handling, fallbacks, or validation for
scenarios
```

```
that can't happen. Trust internal code and framework guarantees. Only validate
at system
boundaries (user input, external APIs).

- Abstractions: Don't create helpers, utilities, or abstractions for one-time
operations. Don't design for hypothetical future requirements. The right amount
of
complexity is the minimum needed for the current task.
```

Avoid focusing on passing tests and hardcoding

Claude can sometimes focus too heavily on making tests pass at the expense of more general solutions, or may use workarounds like helper scripts for complex refactoring instead of using standard tools directly. To prevent this behavior and get solutions that generalize:

Sample prompt

```
Please write a high-quality, general-purpose solution using the standard tools
available. Do not create helper scripts or workarounds to accomplish the task
more
efficiently. Implement a solution that works correctly for all valid inputs,
not just
the test cases. Do not hard-code values or create solutions that only work for
specific
test inputs. Instead, implement the actual logic that solves the problem
generally.

Focus on understanding the problem requirements and implementing the correct
algorithm.
Tests are there to verify correctness, not to define the solution. Provide a
principled
implementation that follows best practices and software design principles.

If the task is unreasonable or infeasible, or if any of the tests are
incorrect, please
inform me rather than working around them. The solution should be robust,
maintainable,
and extendable.
```

Minimizing hallucinations in agentic coding

Claude's latest models are less prone to hallucinations and give more accurate, grounded, intelligent answers based on the code. To encourage this behavior even more and minimize hallucinations:

Sample prompt

```
<investigate_before_answering>
Never speculate about code you have not opened. If the user references a
specific file,
you MUST read the file before answering. Make sure to investigate and read
relevant
files BEFORE answering questions about the codebase. Never make any claims
about code
before investigating unless you are certain of the correct answer - give
grounded and
hallucination-free answers.
</investigate_before_answering>
```

Capability-specific tips

Improved vision capabilities

Claude Opus 4.5 and Claude Opus 4.6 have improved vision capabilities compared to previous Claude models. They perform better on image processing and data extraction tasks, particularly when there are multiple images present in context. These improvements carry over to computer use, where the models can more reliably interpret screenshots and UI elements. You can also use these models to analyze videos by breaking them up into frames.

One technique that has proven effective to further boost performance is to give Claude a crop tool or agent skill. Testing has shown consistent uplift on image evaluations when Claude is able to "zoom" in on relevant regions of an image. Anthropic has created a recipe for the crop tool.

Frontend design

Claude Opus 4.5 and Claude Opus 4.6 build complex, real-world web applications with strong frontend design. However, without guidance, models can default to generic patterns that create what users call the "AI slop" aesthetic. To create distinctive, creative frontends that surprise and delight:

For a detailed guide on improving frontend design, see the blog post on improving frontend design through skills.

For frontend design work outside the API, Claude Design provides a canvas and design tools where Claude generates and iterates on designs interactively.

Here's a system prompt snippet you can use to encourage better frontend design:

Sample prompt for frontend aesthetics

```
<frontend_aesthetics>
You tend to converge toward generic, "on distribution" outputs. In frontend
design, this
creates what users call the "AI slop" aesthetic. Avoid this: make creative,
distinctive
frontends that surprise and delight.

Focus on:
- Typography: Choose fonts that are beautiful, unique, and interesting. Avoid
generic
fonts like Arial and Inter; opt instead for distinctive choices that elevate
the
frontend's aesthetics.
- Color & Theme: Commit to a cohesive aesthetic. Use CSS variables for
consistency.
Dominant colors with sharp accents outperform timid, evenly-distributed
palettes. Draw
from IDE themes and cultural aesthetics for inspiration.
- Motion: Use animations for effects and micro-interactions. Prioritize
CSS-only
solutions for HTML. Use Motion library for React when available. Focus on
high-impact
moments: one well-orchestrated page load with staggered reveals
(animation-delay)
creates more delight than scattered micro-interactions.
- Backgrounds: Create atmosphere and depth rather than defaulting to solid
colors. Layer
CSS gradients, use geometric patterns, or add contextual effects that match the
overall
aesthetic.

Avoid generic AI-generated aesthetics:
- Overused font families (Inter, Roboto, Arial, system fonts)
- Clichéd color schemes (particularly purple gradients on white backgrounds)
- Predictable layouts and component patterns
- Cookie-cutter design that lacks context-specific character

Interpret creatively and make unexpected choices that feel genuinely designed
for the
context. Vary between light and dark themes, different fonts, different
aesthetics. You
still tend to converge on common choices (Space Grotesk, for example) across
```

```
generations. Avoid this: it is critical that you think outside the box!
</frontend_aesthetics>
```

You can also refer to the full skill definition.

Migration considerations

When migrating to current Claude models from earlier generations:

1. **Be specific about desired behavior:** Consider describing exactly what you'd like to see in the output.
2. **Frame your instructions with modifiers:** Adding modifiers that encourage Claude to increase the quality and detail of its output can help better shape Claude's performance. For example, instead of "Create an analytics dashboard", use "Create an analytics dashboard. Include as many relevant features and interactions as possible. Go beyond the basics to create a fully-featured implementation."
3. **Request specific features explicitly:** Animations and interactive elements should be requested explicitly when desired.
4. **Update thinking configuration:** Claude 4.6 and later models use adaptive thinking (`thinking: {type: "adaptive"}`) instead of manual thinking with `budget_tokens`. On Claude 4.7 and later models, setting `budget_tokens` returns a 400 error. Use the effort parameter to control thinking depth.
5. **Migrate away from prefilled responses:** Prefilled responses on the last assistant turn are no longer supported starting with Claude 4.6 models and Claude Mythos Preview. See Migrating away from prefilled responses for detailed guidance on alternatives.
6. **Tune anti-laziness prompting:** If your prompts previously encouraged the model to be more thorough or use tools more aggressively, dial back that guidance. Claude 4.6 models are more proactive and may overtrigger on instructions that were needed for previous models.
7. **Pass thinking blocks back unchanged and keep history append-only:** Append each assistant turn exactly as the API returned it, thinking blocks included. On Claude Fable 5.1, Claude Opus 5.5, Claude Sonnet 5.5, and Claude Haiku 5.5, modifying the conversation before a thinking block results in an error, or in the block being dropped if you opt into that: editing earlier messages, rebuilding `system` or `tools`, or summarizing older turns in place between requests invalidates every later thinking block, so move those changes to mid-conversation system messages and server-side context management. See Keep the conversation history append-only.

For detailed migration steps, see the Migration guide.

Migrating to Claude Sonnet 5.5

See the Claude Sonnet 5.5 migration guide. It covers the breaking changes for code written for Claude Sonnet 5 and the steps from Claude Sonnet 4.6 and earlier, including the removal of manual extended thinking (`budget_tokens`).

Next steps

Prompting Claude Fable 5.1

Behavioral differences and prompting patterns for Claude Fable 5.1, covering effort, task completion, progress updates, thinking blocks, tool-call batching, and writing style.

Prompting Claude Fable 5

Behavioral differences and prompting patterns for Claude Fable 5 and Claude Mythos 5, covering effort, instruction following, long runs, memory, and scaffolding changes.

Prompting Claude Sonnet 5.5

Behavioral differences and prompting patterns for Claude Sonnet 5.5, covering effort calibration, initiative and scope, running without up-front thinking, JSON output on reasoning tasks, progress updates, tool use, mid-turn user messages, coding verification, tool-call handling, visual inputs, and safeguard refusals.

Prompting Claude Sonnet 5

Behavioral differences and prompting patterns for Claude Sonnet 5, covering effort, adaptive thinking defaults, and tool use.

Prompting Claude Opus 5.5

Behavioral differences and prompting patterns for Claude Opus 5.5, covering effort calibration, thinking always on, progress updates, safeguard refusals, and visual inputs.

Prompting Claude Opus 5

Behavioral differences and prompting patterns for Claude Opus 5, covering response verbosity, agentic narration, task scoping, subagent delegation, and self-correction.

Prompting Claude Haiku 5.5

Behavioral differences and prompting patterns for Claude Haiku 5.5, covering effort, search, JSON output with your own tools, early stopping, coding verification, mid-turn user messages, system prompt adherence in chatbots, and safeguard refusals.

Prompt engineering overview

When to use prompt engineering and how to plan your approach before tuning prompts.